Python para Economia — Livro-Texto Completo

Autor: Gerado por IA — Estude, pratique, erre, aprenda.

Livro-texto completo: Python, pandas, visualização, econometria e projetos

Sumário

Python para Economia — Livro-Texto Completo 1
Como Usar Este Livro (Edição Ampliada) 2
Parte I — Fundamentos de Python 2
Capítulo 1 — Introdução 2
Capítulo 2 — Variáveis, Tipos e Operadores 3
Capítulo 3 — Estruturas de Dados 7
Capítulo 4 — Controle de Fluxo 10
Capítulo 5 — Funções 13
Capítulo 6 — List Comprehension e Ferramentas 15
Parte II — pandas para Análise de Dados 17
Capítulo 7 — Introdução ao pandas 17
Capítulo 8 — Seleção e Filtragem 19
Capítulo 9 — Transformação de Dados 20
Capítulo 10 — Datas e Séries Temporais 22
Capítulo 11 — Agrupamento e Junção de Tabelas 24
Capítulo 12 — Acessando Dados Econômicos Reais 25
Parte III — Visualização de Dados 28
Capítulo 13 — matplotlib 28
Capítulo 14 — seaborn 30
Capítulo 15 — Gráficos para Economia 30
Parte IV — Econometria 33
Capítulo 16 — Regressão Linear 33
Capítulo 17 — Diagnóstico e Validação 34
Capítulo 18 — Séries Temporais 35
Capítulo 19 — Dados de Painel 37
Parte V — Projetos 39
Capítulo 20 — Projeto: Análise Macroeconômica do Brasil 39
Capítulo 21 — Projeto: Previsão de Inflação com SARIMA 40
Capítulo 22 — Projeto Final: Estrutura de TCC 40
Apêndices 41
Apêndice A — Guia rápido de consulta 41
Apêndice B — Códigos BCB SGS mais usados 41
Apêndice C — Soluções dos Exercícios 42
Apêndice D — Erros Comuns e Teste Rápido 43

Como Usar Este Livro (Edição Ampliada)

Este livro percorre, em um só volume, o caminho do primeiro `print` à análise econométrica de um projeto de TCC. Ele condensa as cinco apostilas da série (cada uma com tratamento mais extenso): **Parte I** ↔ Apostila 1 (Fundamentos de Python), **Parte II** ↔ Apostila 2 (pandas), **Parte III** ↔ Apostila 3 (Visualização), **Parte IV** ↔ Apostila 4 (Econometria) e **Parte V** ↔ Apostila 5 (Projetos). Use o livro para **consulta e visão de conjunto** e as apostilas para **aprofundar**.

Nesta edição, o fim de cada parte traz um **aprofundamento**: teoria adicional e exemplos *executados* — os resultados impressos vêm da execução do código, com semente fixa, e usam os dados reais deste repositório (`selic_meta.json`, `ipca_12m.json`, `ipca_abertura_grupos.csv`, `taylor_rule_series.csv`). Caixas coloridas indicam o tipo de conteúdo:

Exemplo resolvido — modelo
Um exemplo executado e comentado.

Intuição
A ideia por trás do conceito, sem jargão.

Aprofundando a teoria — modelo
A dedução ou o "por baixo dos panos".

Atenção — erro comum
Um erro clássico.

No mercado brasileiro
Como o conceito aparece no trabalho de um economista.

Atenção — erro comum
Versão do pandas. Os resultados foram gerados com o **pandas 2.2**. O pandas 3.0 tornou padrão o Copy-on-Write (a indexação encadeada deixa de funcionar e o `SettingWithCopyWarning` some), introduziu um tipo `str` dedicado para texto e renomeou alguns aliases de frequência (`"M"` → `"ME"`). Se o seu resultado diferir em algum detalhe, confira `pd.__version__`.

Parte I — Fundamentos de Python

> "A melhor maneira de aprender programação é programar. A melhor maneira de aprender economia é programar modelos econômicos."

Capítulo 1 — Introdução

1.1 Por que Python?

Python se tornou a linguagem padrão em economia por razões objetivas:

1. **Curva de aprendizado suave** — sua sintaxe se aproxima do inglês/português
2. **Ecossistema maduro** — pandas (dados tabulares), numpy (matemática), statsmodels (econometria), matplotlib (gráficos)
3. **Reprodutibilidade** — scripts substituem comandos manuais no Excel/Stata
4. **Gratuito e aberto** — ao contrário de Stata, SAS, EViews

Instituições que usam Python: - Banco Central do Brasil - FMI (Fundo Monetário Internacional) - Banco Mundial - MIT, Harvard, Chicago (cursos de economia) - Quase todas as fintechs e bancos

1.2 Como este livro está organizado

Cada capítulo segue a mesma estrutura:

1. **Conceito** — explicação teórica
2. **Aplicação econômica** — exemplo real
3. **Código** — implementação completa
4. **Exercícios** — para fixação (com soluções no apêndice)

1.3 Como usar este livro

Você precisa de:

1. **Python 3.11+** instalado (já temos no sistema)
2. **Ambiente virtual** ativado (`source ~/economia/.venv/bin/activate`)
3. **Editor de texto** (Cursor AI / VS Code)
4. **Um terminal aberto** para executar os exemplos

Cada bloco de código neste livro foi testado e executa sem erros. Copie, cole, modifique.

Capítulo 2 — Variáveis, Tipos e Operadores

2.1 O que é uma variável?

Uma variável é um nome que armazena um valor na memória do computador. Pense nela como uma caixa com uma etiqueta.

```
pib_brasil_2025 = 2.2  # a caixa se chama "pib_brasil_2025" e contém 2.2
```

Quando você escreve `pib_brasil_2025`, o computador substitui pelo valor `2.2`.

2.2 Tipos de dados fundamentais

Python reconhece quatro tipos básicos automaticamente (tipagem dinâmica).

Inteiros (int)

Números sem casa decimal:

```
ano = 2026
populacao = 214000000
renda_per_capita = 45000
republica_anos = 136  # desde 1889
```

Operações com inteiros produzem inteiros ou floats:

```
print(type(ano))            # <class 'int'>
print(10 / 3)               # 3.333... (divisão sempre retorna float)
print(10 // 3)              # 3 (divisão inteira)
print(10 % 3)               # 1 (resto da divisão)
```

Ponto flutuante (float)

Números decimais:

```
ipca_mensal = 0.16
taxa_selic = 14.75
cambio_usd = 5.45
elasticidade = -0.42
```

Cuidado com precisão:

```
print(0.1 + 0.2)            # 0.30000000000000004 (erro de ponto flutuante)
print(round(0.1 + 0.2, 2))  # 0.3 (sempre arredonde para exibição)
```

Strings (str)
Texto. Sempre entre aspas (simples ou duplas):

```
pais = "Brasil"
sigla = 'BR'
frase = "O PIB cresceu 3.2% em 2025"
vazio = ""
multilinha = """Este é um texto
que ocupa várias linhas"""
```

Caracteres especiais:

```
print("Linha 1\nLinha 2")    # \n = nova linha
print("Coluna1\tColuna2")    # \t = tabulação
```

Booleanos (bool)
Verdadeiro ou falso. Usado em decisões:

```
em_recessao = False
crescendo = True
```

2.3 Operadores aritméticos

```
a = 1000   # principal (R$)
b = 0.01   # taxa de juros (1%)
c = 12     # meses

# Operações básicas
soma = a + a * b * c          # 1120
subtracao = a - 50            # 950
multiplicacao = a * b         # 10
divisao = a / b               # 100000

# Juros compostos
montante = a * (1 + b) ** c   # 1126.83 (mais que 1120 dos juros simples)

# Potenciação
print(10 ** 2)    # 100
print(10 ** 3)    # 1000
print(10 ** -1)   # 0.1

# Ordem das operações (parênteses primeiro)
x = 2 + 3 * 4     # 14
y = (2 + 3) * 4   # 20
```

2.4 Operadores de comparação
Sempre retornam bool:

```
5 == 5      # True (igual)
5 != 3      # True (diferente)
5 > 3       # True
5 < 3       # False
```

```
5 >= 5      # True
5 <= 4      # False
```

Aplicação econômica:

```
pib = -1.5
inflacao = 5.2
desemprego = 11.3

print(pib < 0)                # True (PIB negativo)
print(inflacao > 4.5)         # True
print(desemprego >= 10)       # True
```

2.5 Operadores lógicos

Combinam condições:

```
# AND — ambas verdadeiras
True and True      # True
True and False     # False

# OR — pelo menos uma verdadeira
True or False      # True
False or False     # False

# NOT — inverte
not True           # False
```

Aplicação: detectar estagflação:

```
pib = -0.5
inflacao = 7.2

estagflacao = pib < 0 and inflacao > 5
print(estagflacao)  # True

# Ou recessão ou inflação alta
alerta = pib < 0 or inflacao > 8
```

2.6 Conversão entre tipos

```
# str -> int
int("2026")           # 2026

# int -> str
str(2026)            # "2026"

# str -> float
float("5.45")        # 5.45

# float -> int (trunca!)
int(5.99)            # 5 (cuidado!)

# número -> bool
bool(0)              # False
bool(1)              # True
bool(-1)             # True
bool(0.0)            # False
```

2.7 F-strings (formatação)

Essencial para exibir resultados de forma clara:

```
pais = "Brasil"
pib = 2.2
ano = 2025

# Básico
print(f"{pais} cresceu {pib}% em {ano}")

# Controlar casas decimais
pi = 3.14159265
print(f"{pi:.2f}")    # 3.14
print(f"{pi:.4f}")    # 3.1416

# Formatando porcentagem
print(f"{pib:.1f}%")  # "2.2%"

# Alinhamento
print(f"{'Direita':>10}")   # "    Direita"
print(f"{'Esquerda':<10}")  # "Esquerda  "
print(f"{'Centro':^10}")    # "  Centro  "

# Números grandes
pop = 214000000
print(f"{pop:,}")           # 214,000,000
print(f"{pop:_}")           # 214_000_000
```

2.8 Aplicação econômica completa

```
# Cenário: investimento de R$ 10.000
valor_inicial = 10000
taxa_anual = 0.08  # 8% ao ano
anos = 10

# Cálculo de juros compostos
valor_final = valor_inicial * (1 + taxa_anual) ** anos
rendimento = valor_final - valor_inicial

print(f"Investimento inicial: R$ {valor_inicial:,.2f}")
print(f"Taxa: {taxa_anual * 100:.1f}% ao ano")
print(f"Período: {anos} anos")
print(f"Valor final: R$ {valor_final:,.2f}")
print(f"Rendimento: R$ {rendimento:,.2f}")
print(f"Multiplicador: {(1 + taxa_anual) ** anos:.2f}x")
```

Saída:

```
Investimento inicial: R$ 10,000.00
Taxa: 8.0% ao ano
Período: 10 anos
Valor final: R$ 21,589.25
Rendimento: R$ 11,589.25
Multiplicador: 2.16x
```

Exercícios do Capítulo 2

1. Calcule o montante de R$ 5.000 a 1,5% ao mês por 24 meses
2. Converta US$ 1.000 em reais (câmbio = R$ 5,45) e exiba com duas casas decimais
3. Verifique se as seguintes condições são True/False:

- 10 > 5 and 3 < 2
- 7 == 7 or 4 != 4
- not (5 <= 3)

4. Dada inflação de 4,8% e meta de 3,5%, crie uma variável bool `meta_descumprida`

Capítulo 3 — Estruturas de Dados

3.1 Listas

Lista é uma coleção ordenada e mutável de elementos.

Criando listas

```
# Lista de números
inflacoes = [0.50, 0.70, 0.88, 0.67, 0.58, 0.16]

# Lista de strings
paises = ["Brasil", "Argentina", "Chile", "Peru"]

# Lista mista (evite — prejudica a clareza)
mista = [1, "texto", True, 3.14]

# Lista vazia
vazia = []
vazia = list()
```

Acessando elementos

O índice começa em **zero**:

```
inflacoes[0]    # 0.50  (primeiro elemento)
inflacoes[1]    # 0.70  (segundo elemento)
inflacoes[5]    # 0.16  (sexto elemento)
inflacoes[-1]   # 0.16  (último elemento)
inflacoes[-2]   # 0.58  (penúltimo)

# Erro comum
inflacoes[10]   # IndexError: list index out of range
```

Fatiamento (slicing)

Sintaxe: `lista[inicio:fim:passo]`

```
numeros = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

numeros[0:3]      # [0, 1, 2]  (índices 0, 1, 2 — o 3 não entra)
numeros[:3]       # [0, 1, 2]  (mesma coisa, inicio implícito)
numeros[3:]       # [3, 4, 5, 6, 7, 8, 9]
numeros[2:7]      # [2, 3, 4, 5, 6]
numeros[::2]      # [0, 2, 4, 6, 8]
numeros[1::2]     # [1, 3, 5, 7, 9]
numeros[::-1]     # [9, 8, 7, 6, 5, 4, 3, 2, 1, 0] (inverte)
numeros[-3:]      # [7, 8, 9]  (últimos 3)
numeros[:-3]      # [0, 1, 2, 3, 4, 5, 6]
```

Métodos de lista

```
dados = [1, 2, 3]

dados.append(4)         # [1, 2, 3, 4]  (adiciona ao final)
```

```
dados.insert(0, 0)      # [0, 1, 2, 3, 4]  (insere na posição)
dados.extend([5, 6])    # [0, 1, 2, 3, 4, 5, 6]
dados.pop()             # remove e retorna o último (6)
dados.pop(0)            # remove e retorna o índice 0 (0)
dados.remove(3)         # remove o valor 3
dados.sort()            # ordena crescente
dados.sort(reverse=True)  # ordena decrescente
dados.reverse()         # inverte a ordem
```

Funções nativas para listas

```
numeros = [3, 7, 1, 9, 4, 6]

len(numeros)            # 6 (tamanho)
sum(numeros)            # 30
max(numeros)            # 9
min(numeros)            # 1
sorted(numeros)         # [1, 3, 4, 6, 7, 9]
sorted(numeros, reverse=True)  # [9, 7, 6, 4, 3, 1]
any([False, True])      # True (pelo menos um True)
all([True, True])       # True (todos True)
```

Aplicação econômica

```
# Taxas de crescimento do PIB de 5 países
pibs = [2.2, -1.8, 3.1, 2.8, -0.5]
paises = ["Brasil", "Argentina", "Chile", "Colômbia", "Uruguai"]

print(f"PIB médio: {sum(pibs) / len(pibs):.1f}%")
print(f"Maior PIB: {max(pibs)}% ({paises[pibs.index(max(pibs))]})")
print(f"Menor PIB: {min(pibs)}% ({paises[pibs.index(min(pibs))]})")
print(f"Países em recessão: {sum(1 for p in pibs if p < 0)}")
```

3.2 Tuplas

Tupla é como uma lista, mas **imutável** (não pode ser alterada depois de criada).

```
# Criando tuplas
coordenadas = (-23.5, -46.6)  # latitude, longitude de SP
trimestre = (1, 2, 3)         # meses do primeiro trimestre

# Acesso igual à lista
coordenadas[0]   # -23.5

# Diferença crucial:
coordenadas[0] = -22  # TypeError: 'tuple' object does not support item assignment

# Usos comuns:
# 1. Funções que retornam múltiplos valores (internamente são tuplas)
# 2. Dados que não devem ser modificados
# 3. Chaves de dicionário (listas não podem)
```

3.3 Dicionários

Armazenam pares **chave: valor**. É a estrutura mais importante depois da lista.

Criando dicionários

```
# Dicionário de indicadores
indicadores = {
    "pais": "Brasil",
    "pib": 2.2,
```

```
    "inflacao": 4.5,
    "selic": 14.75,
    "desemprego": 11.3
}

# Outra forma
pibs = dict(Brasil=2.2, Argentina=-1.8, Chile=3.1)
```

Acessando valores

```
indicadores["pais"]       # "Brasil"
indicadores.get("pais")   # "Brasil" (seguro)
indicadores.get("cambio", 0.0)  # 0.0 (valor padrão se não existir)

# Erro comum
indicadores["cambio"]     # KeyError: 'cambio'
```

Adicionar e modificar

```
indicadores["cambio"] = 5.45         # adiciona
indicadores["pib"] = 2.5             # modifica

# Atualizar múltiplos de uma vez
indicadores.update({"pib": 2.8, "inflacao": 4.8})
```

Métodos de dicionário

```
indicadores.keys()     # dict_keys(['pais', 'pib', ...])
indicadores.values()   # dict_values(['Brasil', 2.2, ...])
indicadores.items()    # dict_items([('pais', 'Brasil'), ...])

list(indicadores.keys())   # converter para lista

# Verificar se chave existe
"pib" in indicadores    # True
"cambio" in indicadores # False if not added

# Remover
removido = indicadores.pop("cambio")  # remove e retorna o valor
del indicadores["cambio"]  # remove sem retornar

# Tamanho
len(indicadores)  # número de pares
```

Aplicação econômica

```
# Dicionário aninhado (dados de múltiplos países)
paises = {
    "Brasil": {"pib": 2.2, "inflacao": 4.5, "populacao": 214},
    "Chile":  {"pib": 3.1, "inflacao": 3.8, "populacao": 19},
    "Argentina": {"pib": -1.8, "inflacao": 98.0, "populacao": 46},
}

for pais, dados in paises.items():
    print(f"{pais}: PIB {dados['pib']}%, Inflação {dados['inflacao']}%")
```

3.4 Conjuntos (set)

Coleção não ordenada de elementos únicos:

```python
# Remover duplicatas
paises_visita = ["Brasil", "Chile", "Brasil", "Peru", "Chile"]
unicos = set(paises_visita)  # {"Brasil", "Chile", "Peru"}

# Operações de conjunto
conjunto_a = {1, 2, 3, 4}
conjunto_b = {3, 4, 5, 6}

conjunto_a & conjunto_b  # {3, 4}  (interseção)
conjunto_a | conjunto_b  # {1, 2, 3, 4, 5, 6}  (união)
conjunto_a - conjunto_b  # {1, 2}  (diferença)
```

Exercícios do Capítulo 3

1. Dada a lista `pib = [2.2, -1.8, 3.1, 2.8, -0.5]`, calcule:
   - Quantos países tiveram PIB positivo
   - O PIB médio apenas dos positivos
2. Crie um dicionário com 3 indicadores de 2 países. Calcule a diferença de inflação entre eles.
3. Use fatiamento para extrair:
   - Os 3 primeiros meses de `ipca = [0.5, 0.7, 0.88, 0.67, 0.58, 0.16]`
   - Os 3 últimos
   - A lista invertida
4. Dado o dicionário `moedas = {"USD": 5.45, "EUR": 5.95, "GBP": 6.90, "ARS": 0.006}`:
   - Qual moeda tem a maior taxa?
   - Quantas moedas estão abaixo de R$ 1,00?
   - Adicione "JPY": 0.036

Capítulo 4 — Controle de Fluxo

4.1 Condicionais (if/elif/else)

Permitem que o programa tome decisões.

Estrutura básica

```python
if condicao:
    # bloco executado se condicao for True
    pass
elif outra_condicao:
    # bloco executado se a primeira for False e esta for True
    pass
else:
    # bloco executado se todas as anteriores forem False
    pass
```

Exemplos econômicos

```python
# Classificação de inflação
inflacao = 7.2

if inflacao > 10:
    print("Hiperinflação")
elif inflacao > 6:
    print("Inflação alta")
```

```python
elif inflacao > 3:
    print("Inflação moderada")
elif inflacao > 0:
    print("Inflação controlada")
else:
    print("Deflação")

# Aninhamento
pib = -0.8
desemprego = 12.5

if pib < 0:
    if desemprego > 10:
        print("Recessão com alto desemprego")
    else:
        print("Recessão")
elif pib > 0:
    print("Crescimento")
```

Operador ternário

```python
# Forma compacta de if/else
status = "Crescendo" if pib > 0 else "Recessão"

# Equivalente a:
if pib > 0:
    status = "Crescendo"
else:
    status = "Recessão"
```

Match/case (Python 3.10+)

```python
# Similar ao switch de outras linguagens
codigo_pais = "BR"

match codigo_pais:
    case "BR":
        nome = "Brasil"
    case "AR":
        nome = "Argentina"
    case "CL":
        nome = "Chile"
    case _:
        nome = "Outro"
```

4.2 Loop for

Percorre elementos de uma sequência.

For com listas

```python
inflacoes = [0.5, 0.7, 0.88, 0.67]

for valor in inflacoes:
    print(f"IPCA: {valor}%")

# Com enumerate (índice + valor)
for i, valor in enumerate(inflacoes):
    print(f"Mês {i + 1}: {valor}%")
```

For com dicionários

```
pibs = {"Brasil": 2.2, "Chile": 3.1, "Peru": 2.5}

# Apenas chaves
for pais in pibs:
    print(pais)

# Chaves e valores
for pais, valor in pibs.items():
    print(f"{pais}: {valor}%")
```

For com range()

```
# range(5) → 0, 1, 2, 3, 4
for i in range(5):
    print(i)

# range(inicio, fim)
for mes in range(1, 13):
    print(f"Mês {mes}")

# range(inicio, fim, passo)
for ano in range(2000, 2026, 5):
    print(f"Ano {ano}")
```

Aplicação: juros compostos mês a mês

```
saldo = 1000
taxa = 0.01  # 1% ao mês

print("Mês   Saldo")
print("=" * 12)
for mes in range(1, 13):
    saldo *= (1 + taxa)
    print(f"{mes:3d}   R$ {saldo:>8.2f}")
```

4.3 Loop while

Repete enquanto a condição for verdadeira.

```
# Mesmo exemplo com while
saldo = 1000
taxa = 0.01
mes = 1

while mes <= 12:
    saldo *= (1 + taxa)
    print(f"Mês {mes}: R$ {saldo:.2f}")
    mes += 1
```

Cuidado com loop infinito: sempre garanta que a condição se torne False em algum momento.

```
# ERRADO — loop infinito
# while True:
#     print("Isso nunca para")

# CERTO — tem condição de parada
saldo = 1000
anos = 0
while saldo < 2000:
```

```
    saldo *= 1.05
    anos += 1
print(f"Leva {anos} anos para dobrar a R$ {saldo:.2f}")
```

4.4 Break e Continue

```
# break — interrompe o loop
for valor in [0.5, 0.7, 1.2, 0.88, 0.67]:
    if valor > 1.0:
        print("Alta detectada! Parando.")
        break
    print(f"IPCA: {valor}%")

# continue — pula para a próxima iteração
for valor in [0.5, 0.7, 0.88, 0.67, 0.58, 0.16]:
    if valor < 0.3:
        continue  # pula meses de deflação
    print(f"IPCA: {valor}%")
```

Exercícios do Capítulo 4

1. Dada uma lista de PIBs anuais, classifique cada ano como "Recessão" (< 0), "Crescimento fraco" (0-2), "Crescimento forte" (> 2). Use if/elif/else.
2. Use for para simular R$ 5.000 a 0,8% ao mês por 36 meses. Mostre o saldo a cada 6 meses.
3. Com while, descubra quantos meses leva para R$ 1.000 virar R$ 2.000 a 1,2% ao mês.
4. Percorra uma lista de inflações e pare ao encontrar a primeira acima de 1%.

Capítulo 5 — Funções

5.1 Definindo funções

```
def nome_da_funcao(parametro1, parametro2):
    """Docstring: explica o que a função faz"""
    resultado = parametro1 + parametro2
    return resultado
```

5.2 Função simples

```
def media_aritmetica(dados):
    """Calcula a média aritmética de uma lista"""
    return sum(dados) / len(dados)

inflacoes = [0.5, 0.7, 0.88, 0.67, 0.58, 0.16]
print(f"Média: {media_aritmetica(inflacoes):.2f}%")
```

5.3 Múltiplos parâmetros

```
def juros_compostos(principal, taxa, periodos):
    """Calcula montante final de juros compostos"""
    return principal * (1 + taxa) ** periodos

montante = juros_compostos(1000, 0.01, 12)
print(f"Montante: R$ {montante:.2f}")
```

5.4 Parâmetros com valor padrão

```
def pib_per_capita(pib_total, populacao, unidade="mil"):
    """Calcula PIB per capita"""
    resultado = pib_total / populacao
    if unidade == "mil":
        return resultado / 1000
    elif unidade == "unidade":
        return resultado
    return resultado

# Chamadas possíveis:
pib_per_capita(2e12, 214e6)              # usa "mil" (padrão)
pib_per_capita(2e12, 214e6, "unidade")   # explicita
pib_per_capita(2e12, 214e6, unidade="unidade")  # nomeando
```

5.5 Retorno múltiplo

```
def estatisticas(dados):
    """Retorna média, mínimo e máximo"""
    media = sum(dados) / len(dados)
    minimo = min(dados)
    maximo = max(dados)
    return media, minimo, maximo

inf = [0.5, 0.7, 0.88, 0.67, 0.58, 0.16]
media, min_inf, max_inf = estatisticas(inf)
print(f"Média: {media:.2f}, Mín: {min_inf:.2f}, Máx: {max_inf:.2f}")
```

5.6 Função como argumento

```
def aplicar_a_cada(lista, funcao):
    """Aplica uma função a cada elemento da lista"""
    return [funcao(x) for x in lista]

def dobrar(x):
    return x * 2

resultado = aplicar_a_cada([1, 2, 3], dobrar)
print(resultado)  # [2, 4, 6]
```

5.7 Funções lambda (anônimas)

```
# Lambda é uma função sem nome, em uma linha
quadrado = lambda x: x ** 2
print(quadrado(5))  # 25

# Útil com sorted, filter, map
paises = [("Brasil", 2.2), ("Argentina", -1.8), ("Chile", 3.1)]
ordenado = sorted(paises, key=lambda x: x[1])  # ordena pelo PIB
print(ordenado)  # [("Argentina", -1.8), ("Brasil", 2.2), ("Chile", 3.1)]
```

Exercícios do Capítulo 5

1. Crie uma função `inflacao_acumulada` que recebe uma lista de taxas mensais e retorna a acumulada.
2. Crie `curva_laffer` que recebe alíquota e retorna arrecadação = alíquota * base * (1 - alíquota). Encontre a alíquota ótima.
3. Crie `classificar_pais` que recebe PIB e inflação e retorna:
   - "Bom" se PIB > 2 e inflação < 5

- "Atenção" se um dos dois está ruim
- "Crítico" se ambos estão ruins

4. Use `sorted` com lambda para ordenar países por inflação decrescente.

Capítulo 6 — List Comprehension e Ferramentas

6.1 List comprehension

Cria listas de forma concisa:

```
# [expressao for item in iteravel if condicao]

# Básico
quadrados = [x ** 2 for x in range(10)]
# [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]

# Com filtro
pares = [x for x in range(20) if x % 2 == 0]

# Aplicação econômica
inflacoes = [0.5, 0.7, 0.88, 0.67, 0.58, 0.16]
acima_media = [x for x in inflacoes if x > sum(inflacoes) / len(inflacoes)]

# Transformação
taxas_reais = [selic - ipca for selic, ipca in zip(selic_list, ipca_list)]

# Dicionário comprehension
quadrados_dict = {x: x ** 2 for x in range(5)}
# {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}
```

6.2 zip — juntar listas

```
paises = ["Brasil", "Chile", "Argentina"]
pibs = [2.2, 3.1, -1.8]

for pais, pib in zip(paises, pibs):
    print(f"{pais}: {pib}%")

# Criar dicionário a partir de duas listas
dic = dict(zip(paises, pibs))
```

6.3 enumerate — índice + valor

```
for i, valor in enumerate(inflacoes, start=1):
    print(f"Mês {i}: {valor}%")
```

6.4 map e filter

```
# map — aplica função a cada elemento
def ao_quadrado(x):
    return x ** 2

list(map(ao_quadrado, [1, 2, 3]))  # [1, 4, 9]
list(map(lambda x: x ** 2, [1, 2, 3]))  # [1, 4, 9]

# filter — filtra elementos
list(filter(lambda x: x > 0, [-1, 2, -3, 4]))  # [2, 4]
```

6.5 any e all

```
# any — True se pelo menos um elemento for True
# all — True se todos os elementos forem True

dados = [True, False, True]
any(dados)  # True
all(dados)  # False

# Aplicação: verificar se algum país está em recessão
pibs = [2.2, -1.8, 3.1]
any(p < 0 for p in pibs)  # True
all(p > 0 for p in pibs)  # False
```

Exercícios do Capítulo 6

1. Use list comprehension para criar lista de PIBs acima da média.
2. Use `zip` para criar dicionário de paises -> inflação.
3. Use `enumerate` para exibir "País 1: Brasil (2.2%)".
4. Use `filter` para extrair apenas valores positivos de uma lista.

Aprofundando a teoria — variáveis são rótulos: identidade, igualdade e aliasing
Em `x = [1, 2]`, o Python cria o objeto lista e **amarra o nome** `x` a ele; `y = x` não copia nada, cria *outro rótulo* para o mesmo objeto. `a == b` pergunta se os *valores* são iguais; `a is b`, se são o *mesmo objeto*. Com tipos **mutáveis** (listas, dicionários), alterar por um rótulo altera "os dois" — fonte clássica de bugs, e a razão de existirem `.copy()` e `copy.deepcopy`.

```
import copy
a = [1, 2, 3]
b = a                   # outro rótulo para o MESMO objeto
c = list(a)             # cópia: novo objeto com os mesmos valores
b.append(4)
print(f"a = {a} | b = {b} | c = {c} | a is b: {a is b} | a is c: {a is c}")

base = [["Brasil", 2.2], ["Chile", 3.1]]
rasa, funda = list(base), copy.deepcopy(base)
base[0][1] = 9.9
print("Após alterar base[0][1] -> cópia rasa:", rasa, "| cópia profunda:", funda)
```

```
a = [1, 2, 3, 4] | b = [1, 2, 3, 4] | c = [1, 2, 3] | a is b: True | a is c: False
Após alterar base[0][1] -> cópia rasa: [['Brasil', 9.9], ['Chile', 3.1]] | cópia profunda:
[['Brasil', 2.2], ['Chile', 3.1]]
```

Aprofundando a teoria — dinheiro em ponto flutuante: Decimal, arredondamento e tolerância
O `float` é binário e aproximado: `0.1 + 0.2` não é exatamente `0.3`. Para valores monetários onde os centavos precisam fechar, use `decimal.Decimal` (ou inteiros em centavos). O `round()` do Python usa o **arredondamento bancário** (empates vão ao par), e compara-se `float` com tolerância (`math.isclose`), nunca com ==.

```
from decimal import Decimal, ROUND_HALF_UP
import math
acum_f, acum_d = 0.0, Decimal("0")
for _ in range(10):
    acum_f += 0.1; acum_d += Decimal("0.1")
print("Dez vezes +0,10 -> float:", acum_f, "| Decimal:", acum_d)
print("round(0.5), round(1.5), round(2.5):", round(0.5), round(1.5), round(2.5), "|
round(2.675, 2) =", round(2.675, 2))
```

```
print("Decimal('2.675') arredondado (ROUND_HALF_UP):",
Decimal("2.675").quantize(Decimal("0.01"), rounding=ROUND_HALF_UP))
print("math.isclose(0.1 + 0.2, 0.3):", math.isclose(0.1 + 0.2, 0.3))
print("-7 // 2 =", -7 // 2, "| -7 % 2 =", -7 % 2, "| int(-7 / 2) =", int(-7 / 2), "| '' or
'padrão' ->", repr("" or "padrão"))
```

```
Dez vezes +0,10 -> float: 0.9999999999999999 | Decimal: 1.0
round(0.5), round(1.5), round(2.5): 0 2 2 | round(2.675, 2) = 2.67
Decimal('2.675') arredondado (ROUND_HALF_UP): 2.68
math.isclose(0.1 + 0.2, 0.3): True
-7 // 2 = -4 | -7 % 2 = 1 | int(-7 / 2) = -3 | '' or 'padrão' -> 'padrão'
```

Atenção — erro comum
`taxa or padrao` trata `0.0` como "ausente", porque zero é falso. Se zero é um valor legítimo (taxa de 0%), compare com `None`: `if taxa is None: taxa = padrao`. E `lista = lista.sort()` apaga a lista (`sort()` devolve `None`).

Parte II — pandas para Análise de Dados

Capítulo 7 — Introdução ao pandas

7.1 O que é pandas?

pandas é a biblioteca para manipulação de dados tabulares. Os dois objetos principais:

- **Series**: uma coluna (vetor com índice)
- **DataFrame**: tabela completa (várias colunas)

```
import pandas as pd
import numpy as np
```

7.2 Criando DataFrames

A partir de dicionário

```
dados = {
    "pais": ["Brasil", "Argentina", "Chile", "Colômbia", "Peru"],
    "pib_2024": [3.2, -1.8, 2.5, 1.8, 3.0],
    "inflacao": [4.5, 98.0, 3.8, 7.2, 2.0],
    "populacao_milhoes": [214, 46, 19, 52, 34],
    "regiao": ["América do Sul"] * 5
}

df = pd.DataFrame(dados)
print(df)
```

Saída:

```
       pais  pib_2024  inflacao  populacao_milhoes           regiao
0    Brasil       3.2       4.5                214  América do Sul
1  Argentina      -1.8      98.0                 46  América do Sul
2     Chile       2.5       3.8                 19  América do Sul
3  Colombia       1.8       7.2                 52  América do Sul
4      Peru       3.0       2.0                 34  América do Sul
```

A partir de arquivos

```
# CSV
df = pd.read_csv("dados.csv")
df = pd.read_csv("dados.csv", decimal=",", sep=";")  # formato brasileiro

# Excel
df = pd.read_excel("dados.xlsx", sheet_name="Planilha1")

# Clipboard (copie do Excel e rode)
# df = pd.read_clipboard()
```

A partir de API do BCB

```
from bcb import sgs

# Códigos SGS: IPCA=433, SELIC=11, Desemprego=24369, Câmbio=1
df = sgs.get({"IPCA": 433, "SELIC": 11}, start="2020-01-01")
print(df.head())
```

7.3 Explorando o DataFrame

```
df.head()             # primeiras 5 linhas
df.head(10)           # primeiras 10
df.tail(3)            # últimas 3
df.sample(5)          # 5 amostras aleatórias

df.shape              # (linhas, colunas)
df.info()             # tipos + nulos + memória
df.describe()         # estatísticas descritivas
df.describe(include="object")  # para colunas de texto

df.dtypes             # tipo de cada coluna
df.columns            # nomes das colunas
df.index              # índice
df.values             # matriz numpy subjacente

df["pais"].unique()          # valores únicos
df["pais"].nunique()         # quantidade de valores únicos
df["pais"].value_counts()    # contagem de cada valor

df.isnull().sum()     # valores nulos por coluna
df.duplicated().sum() # linhas duplicadas
```

7.4 O atributo dtype

Cada coluna tem um tipo:

dtype	Significado
`int64`	Inteiro
`float64`	Decimal
`object`	Texto (string)
`datetime64`	Data/hora
`bool`	Booleano

```
# Converter tipos
df["pib_2024"] = df["pib_2024"].astype(float)
df["data"] = pd.to_datetime(df["data"])
```

Exercícios do Capítulo 7

1. Crie um DataFrame com 5 países e 4 indicadores.
2. Use `describe()` e interprete as estatísticas.
3. Use `value_counts()` em uma coluna categórica.
4. Verifique se há valores nulos.

Capítulo 8 — Seleção e Filtragem

8.1 Selecionando colunas

```
# Uma coluna (retorna Series)
df["pais"]
df.pais  # atalho (funciona se nome não tiver espaço)

# Várias colunas (retorna DataFrame)
df[["pais", "pib_2024"]]
```

8.2 Selecionando linhas por posição (iloc)

```
df.iloc[0]        # primeira linha
df.iloc[-1]       # última linha
df.iloc[0:3]      # linhas 0, 1, 2
df.iloc[[0, 2]]   # linhas 0 e 2

df.iloc[0, 1]     # linha 0, coluna 1
df.iloc[0:3, 0:2] # linhas 0-2, colunas 0-1
df.iloc[:, 0:2]   # todas linhas, colunas 0-1
```

8.3 Selecionando linhas por rótulo (loc)

```
# Primeiro: definir índice
df.index = ["BR", "AR", "CL", "CO", "PE"]

df.loc["BR"]          # linha Brasil
df.loc["BR":"CL"]     # BR, AR, CL (inclusive)
df.loc[["BR", "CL"]]  # BR e CL
df.loc[:, "pais"]     # todas linhas, coluna "pais"
df.loc["BR", "pib_2024"]  # valor específico
```

8.4 Filtrando linhas (a operação MAIS importante)

```
# Sintaxe: df[condicao]
# A condição é uma Series de True/False

# Filtro simples
df[df["pib_2024"] > 2.0]           # PIB > 2%
df[df["inflacao"] < 10]            # inflação baixa
df[df["pais"] == "Brasil"]         # apenas Brasil

# Múltiplas condições
df[(df["pib_2024"] > 0) & (df["inflacao"] < 10)]
df[(df["pib_2024"] < 0) | (df["inflacao"] > 10)]

# isin — vários valores
df[df["pais"].isin(["Brasil", "Chile"])]

# between — intervalo
```

```python
df[df["populacao_milhoes"].between(20, 100)]

# str.contains — texto
df[df["pais"].str.contains("il")]  # "Brasil", "Chile"

# Negação
df[~(df["pais"] == "Brasil")]

# query (sintaxe alternativa, útil em strings longas)
df.query("pib_2024 > 0 and inflacao < 10")
```

8.5 Como funciona por baixo

```python
mascara = df["pib_2024"] > 2.0
print(mascara)
# 0     True
# 1    False
# 2     True
# 3    False
# 4     True

df[mascara]  # aplica o filtro
```

Exercícios do Capítulo 8

1. Selecione apenas países com inflação menor que 5%.
2. Selecione países com PIB positivo E inflação menor que 10%.
3. Use `iloc` para extrair as últimas 3 linhas.
4. Use `query` para filtrar países com PIB > 2 e população > 30 milhões.

Capítulo 9 — Transformação de Dados

9.1 Criando colunas

```python
# Aritmética simples
df["pib_per_capita"] = df["pib_2024"] / df["populacao_milhoes"] * 1000

# Transformação
df["log_pop"] = np.log(df["populacao_milhoes"])
df["pib_categoria"] = df["pib_2024"].apply(lambda x: "Alto" if x > 2 else "Baixo")

# with columns (útil para múltiplas)
df = df.assign(
    pib_dobro=df["pib_2024"] * 2,
    inflacao_decimal=df["inflacao"] / 100
)
```

9.2 Apply — aplicando funções

```python
# Apply em coluna
def classificar_inflacao(x):
    if x > 10: return "Hiper"
    elif x > 5: return "Alta"
    elif x > 2: return "Moderada"
    else: return "Baixa"

df["categoria_inflacao"] = df["inflacao"].apply(classificar_inflacao)
```

```
# Apply em DataFrame (por linha)
df[["pib_2024", "inflacao"]].apply(lambda x: x.mean(), axis=1)  # média por linha
```

9.3 Renomeando colunas

```
df.rename(columns={"pib_2024": "crescimento_pib"}, inplace=True)
df.columns = [c.upper() for c in df.columns]  # tudo maiúsculo
```

9.4 Removendo colunas e linhas

```
# Remover colunas
df.drop("coluna_que_nao_quero", axis=1, inplace=True)
df.drop(["col1", "col2"], axis=1, inplace=True)

# Remover linhas
df.drop([0, 2], inplace=True)  # remove índices 0 e 2
df.drop(df[df["pib_2024"] < 0].index, inplace=True)  # remove recessão
```

9.5 Valores nulos

```
# Verificar
df.isnull().sum()
df.isna().sum()  # mesmo que isnull

# Remover
df.dropna()                          # remove qualquer linha com nulo
df.dropna(subset=["pib_2024"])       # remove só se coluna específica for nula
df.dropna(thresh=3)                  # remove se tiver menos de 3 não-nulos

# Preencher
df.fillna(0)                         # com zero
df.fillna(df.mean())                 # com a média da coluna
df["coluna"].fillna(method="ffill") # último valor válido (forward fill)
df["coluna"].fillna(method="bfill") # próximo valor válido (backward fill)
```

9.6 Ordenação

```
# Por valor
df.sort_values("pib_2024")                      # crescente
df.sort_values("pib_2024", ascending=False)     # decrescente
df.sort_values(["regiao", "pib_2024"])          # múltiplas colunas

# Por índice
df.sort_index()
```

9.7 Amostragem

```
df.sample(5)             # 5 amostras aleatórias
df.sample(frac=0.1)      # 10% das linhas
df.sample(5, random_state=42)  # reprodutível
```

Exercícios do Capítulo 9

1. Crie coluna `pib_per_capita = pib / populacao * 1000`.
2. Classifique países: "Grande" se população > 50 milhões, "Médio" se > 20, "Pequeno" caso contrário.
3. Preencha valores nulos com a média da coluna.
4. Ordene o DataFrame por inflação decrescente.

Capítulo 10 — Datas e Séries Temporais

10.1 Convertendo para datetime

```
df["data"] = pd.to_datetime(df["data"])
```

Estratégias comuns de conversão:

```
# Formatos comuns
pd.to_datetime("2024-01-15")              # yyyy-mm-dd
pd.to_datetime("15/01/2024", dayfirst=True)  # dd/mm/yyyy
pd.to_datetime("Jan 15 2024")
pd.to_datetime("20240115", format="%Y%m%d")  # formato explícito

# Erro: misturar formatos
# use errors="coerce" para converter o que der e virar NaT no resto
pd.to_datetime(coluna, errors="coerce")
```

10.2 Extraindo componentes

```
df["ano"] = df["data"].dt.year
df["mes"] = df["data"].dt.month
df["dia"] = df["data"].dt.day
df["trimestre"] = df["data"].dt.quarter
df["semestre"] = df["data"].dt.semester
df["semana"] = df["data"].dt.isocalendar().week
df["dia_semana"] = df["data"].dt.dayofweek  # 0=segunda, 6=domingo
df["nome_mes"] = df["data"].dt.month_name("pt_BR")
```

10.3 Indexando por data

```
df.set_index("data", inplace=True)
df = df.sort_index()  # sempre ordene o índice

# Fatiamento temporal
df.loc["2024"]                          # ano inteiro
df.loc["2024-01":"2024-06"]             # primeiro semestre
df.loc["2024-01-15":]                   # a partir de 15/01/2024

# Condições com datas
df[df.index.year >= 2023]
df[df.index.month == 12]                # apenas dezembros
```

10.4 Reamostragem (resample)

Muda a frequência dos dados.

```
# Agregar por período
df.resample("Y").mean()       # anual (year)
df.resample("Q").mean()       # trimestral (quarter)
df.resample("M").mean()       # mensal (month)
df.resample("W").mean()       # semanal (week)
df.resample("D").mean()       # diário

# Agregações diferentes
df.resample("Y").agg({
    "pib": "mean",
    "inflacao": "max",
```

```
    "desemprego": "last"
})
```

10.5 Lag, diferença e variação percentual

Essenciais para séries temporais.

```
# Lag (valor do período anterior)
df["pib_lag1"] = df["pib"].shift(1)
df["pib_lag2"] = df["pib"].shift(2)

# Diferença absoluta
df["pib_diff"] = df["pib"].diff()       # pib_t - pib_{t-1}

# Variação percentual
df["pib_pct"] = df["pib"].pct_change() * 100  # em %

# Média móvel
df["pib_mm3"] = df["pib"].rolling(window=3).mean()
df["pib_mm12"] = df["pib"].rolling(window=12).mean()

# Desvio padrão móvel
df["pib_vol"] = df["pib"].rolling(window=12).std()
```

10.6 Aplicação completa

```
from bcb import sgs

# Baixar IPCA de 2010 a 2026
ipca = sgs.get({"IPCA": 433}, start="2010-01-01")
ipca.columns = ["ipca"]
ipca.index = pd.to_datetime(ipca.index)

# Criar features temporais
df = ipca.copy()
df["ano"] = df.index.year
df["mes"] = df.index.month
df["trimestre"] = df.index.quarter

# Lags e médias
df["ipca_lag1"] = df["ipca"].shift(1)
df["ipca_mm12"] = df["ipca"].rolling(12).mean()
df["ipca_acum"] = (1 + df["ipca"] / 100).cumprod() - 1
df["ipca_acum"] *= 100

print(df.tail(12))
```

Exercícios do Capítulo 10

1. Baixe SELIC de 2015 a 2026. Calcule média móvel de 6 e 12 meses.
2. Baixe IPCA e calcule inflação acumulada em 12 meses (`.rolling(12).sum()`).
3. Crie coluna com a diferença da SELIC mês a mês.
4. Reamostre IPCA para frequência trimestral com média.
5. Extraia apenas meses de dezembro de todos os anos e calcule a inflação média de dezembro.

Capítulo 11 — Agrupamento e Junção de Tabelas

11.1 Groupby — tabelas dinâmicas

```
# Criando dados para exemplo
np.random.seed(42)
dados = {
    "ano": np.repeat(range(2020, 2025), 3),
    "pais": ["Brasil", "Chile", "Argentina"] * 5,
    "pib": np.random.uniform(-2, 5, 15),
    "inflacao": np.random.uniform(2, 15, 15)
}
df = pd.DataFrame(dados)

# Agregar por uma coluna
df.groupby("pais")["pib"].mean()
df.groupby("pais")["inflacao"].agg(["mean", "std", "min", "max"])

# Múltiplas colunas
df.groupby(["ano", "pais"])["pib"].mean()

# Várias agregações simultâneas
df.groupby("pais").agg({
    "pib": ["mean", "std", "count"],
    "inflacao": "mean"
})

# Achatando colunas
agrupado = df.groupby("pais").agg(
    pib_medio=("pib", "mean"),
    pib_std=("pib", "std"),
    inflacao_media=("inflacao", "mean")
)
```

11.2 Merge — juntando tabelas

Equivalente ao JOIN do SQL ou PROCX do Excel.

```
pib_df = pd.DataFrame({
    "pais": ["Brasil", "Argentina", "Chile", "Peru"],
    "pib": [2.2, -1.8, 3.1, 2.5],
    "ano": [2024, 2024, 2024, 2024]
})

inf_df = pd.DataFrame({
    "pais": ["Brasil", "Chile", "Peru", "Uruguai"],
    "inflacao": [4.5, 3.8, 2.0, 5.5],
    "ano": [2024, 2024, 2024, 2024]
})

# Inner (só interseção)
inner = pd.merge(pib_df, inf_df, on="pais")  # 3 países: Brasil, Chile, Peru

# Left (tudo da esquerda)
left = pd.merge(pib_df, inf_df, on="pais", how="left")

# Right (tudo da direita)
right = pd.merge(pib_df, inf_df, on="pais", how="right")

# Outer (tudo de ambos)
```

```python
outer = pd.merge(pib_df, inf_df, on="pais", how="outer")

# Quando as chaves têm nomes diferentes
pib_df2 = pib_df.rename(columns={"pais": "nome_pais"})
merged = pd.merge(pib_df2, inf_df, left_on="nome_pais", right_on="pais")

# Múltiplas chaves
pd.merge(pib_df, inf_df, on=["pais", "ano"])
```

11.3 Concatenando DataFrames

```python
# Empilhar verticalmente (mais linhas)
df1 = pd.DataFrame({"pais": ["A", "B"], "pib": [1, 2]})
df2 = pd.DataFrame({"pais": ["C", "D"], "pib": [3, 4]})
pd.concat([df1, df2], ignore_index=True)

# Empilhar horizontalmente (mais colunas)
pd.concat([df1, df2], axis=1)
```

Exercícios do Capítulo 11

1. Crie dois DataFrames (PIB e inflação de 5 países) e faça inner, left, right e outer merge.
2. Use groupby para calcular PIB médio por região (crie uma coluna "regiao").
3. Calcule o desvio padrão do PIB por ano.
4. Junte dados de PIB e inflação por país e ano, depois calcule a correlação.

Capítulo 12 — Acessando Dados Econômicos Reais

12.1 BCB SGS (Sistema Gerenciador de Séries Temporais)

```python
from bcb import sgs

# Códigos comuns
codigos = {
    "IPCA": 433,
    "SELIC": 11,
    "CAMBIO": 1,
    "DESEMPREGO": 24369,
    "PIB_MENSAL": 4380,
}

# Baixar múltiplos
dados = sgs.get(codigos, start="2020-01-01")
```

Onde encontrar códigos: - https://www3.bcb.gov.br/sgspub/ - Pesquise por nome da série

12.2 Banco Mundial (WDI)

```python
# Via pandas_datareader (precisa instalar)
# pip install pandas_datareader

from pandas_datareader import wb

# Indicadores comuns
# NY.GDP.PCAP.PP.KD: PIB per capita PPP
# FP.CPI.TOTL.ZG: Inflação
# SL.UEM.TOTL.ZS: Desemprego
```

```
indicadores = ["NY.GDP.PCAP.PP.KD", "FP.CPI.TOTL.ZG", "SL.UEM.TOTL.ZS"]
paises = ["BR", "CL", "AR", "CO", "PE"]

df = wb.download(country=paises, indicator=indicadores,
                 start=2010, end=2024)
```

12.3 Ipeadata (dados brasileiros históricos)

```
# Requer requests + parsing manual
# Alternativa: usar arquivos do site do Ipeadata
```

12.4 FMI (IMF Data)

```
# Via imfpy ou acesso direto ao JSON
```

Exercícios do Capítulo 12

1. Baixe IPCA, SELIC e PIB mensal do BCB de 2010 a 2026.
2. Crie um DataFrame único com os três.
3. Calcule juro real (SELIC - IPCA acumulado 12m).
4. Salve em CSV com ponto e vírgula e vírgula como decimal.

> **Aprofundando a teoria — por que pandas é rápido, e como ler um arquivo "à brasileira"**
> Um array NumPy guarda valores **contíguos** e de **um único tipo** a operação sobre o array inteiro (**vetorização**) delega o laço a código compilado. O pandas constrói tabelas sobre esses arrays. Para ler arquivos brasileiros (separador `;`, vírgula decimal, ponto de milhar, datas `dd/mm/aaaa`, `N/D` como ausente), cada convenção tem um parâmetro de `read_csv`:

```
from io import StringIO
csv_br = """Data;Estado;PIB (R$ mi);Obs
31/03/2025;SP;"2.345,6";ok
31/03/2025;RJ;"1.020,5";
30/06/2025;SP;"2.401,0";revisado
30/06/2025;RJ;N/D;ok"""
lido = pd.read_csv(StringIO(csv_br), sep=";", decimal=",", thousands=".",
parse_dates=["Data"], dayfirst=True, na_values=["N/D"])
print(lido.to_string(), lido.dtypes.to_dict(), sep="\n")

def ler_bcb(caminho):
    d = pd.read_json(caminho, dtype=False)
    d["data"] = pd.to_datetime(d["data"], format="%d/%m/%Y")
    return d.set_index("data")["valor"].astype(float)

sel_l, ip_l = ler_bcb("selic_meta.json"), ler_bcb("ipca_12m.json")
gr_l = pd.read_csv("ipca_abertura_grupos.csv", parse_dates=["data"], index_col="data")
tay_l = pd.read_csv("taylor_rule_series.csv", parse_dates=["data"], index_col="data")
print(f"\nSelic diária: {len(sel_l)} dias até {sel_l.index.max():%d/%m/%Y} | IPCA 12m:
{len(ip_l)} meses (último {ip_l.iloc[-1]:.2f}%)")
```

```
        Data Estado  PIB (R$ mi)       Obs
0 2025-03-31     SP       2345.6        ok
1 2025-03-31     RJ       1020.5       NaN
2 2025-06-30     SP       2401.0  revisado
3 2025-06-30     RJ          NaN        ok
{'Data': dtype('<M8[ns]'), 'Estado': dtype('O'), 'PIB (R$ mi)': dtype('float64'), 'Obs':
dtype('O')}
```

```
Selic diária: 1355 dias até 16/09/2026 | IPCA 12m: 43 meses (último 4.44%)
```

Exemplo resolvido — merge: a explosão de linhas e o `merge_asof` entre frequências diferentes
O `merge` cria uma linha para *cada combinação* de linhas com a mesma chave: com chaves repetidas nas duas tabelas, o total **multiplica** — um dos erros silenciosos mais graves em análise de dados. E para unir uma série diária a uma mensal, o `merge_asof` associa a observação *mais recente anterior*:

```
tab_a = pd.DataFrame({"pais": ["BR", "BR", "CL"], "pib": [3.1, 2.2, 3.1]})
tab_b = pd.DataFrame({"pais": ["BR", "BR", "CL"], "evento": ["eleição", "chuvas",
"plebiscito"]})
print(f"a tem {len(tab_a)} linhas, b tem {len(tab_b)}, merge tem {len(tab_a.merge(tab_b,
on='pais'))} (BR aparece 2 x 2 = 4 vezes)")

casado = pd.merge_asof(sel_l.rename("selic").reset_index(),
ip_l.rename("ipca_12m").reset_index(), on="data", direction="backward")
ex_post = (1 + casado["selic"] / 100) / (1 + casado["ipca_12m"] / 100) - 1
print(casado.dropna().tail(2).to_string(index=False))
print(f"Juro real ex-post (Fisher) no último dia: {ex_post.iloc[-1] * 100:.2f}% a.a.")
```

```
a tem 3 linhas, b tem 3, merge tem 5 (BR aparece 2 x 2 = 4 vezes)
      data  selic  ipca_12m
2026-09-15   14.0      4.44
2026-09-16   14.0      4.44
Juro real ex-post (Fisher) no último dia: 9.15% a.a.
```

Aprofundando a teoria — dados faltantes: a causa decide o tratamento (MCAR, MAR, MNAR) e a natureza da série também
Os dados podem faltar **ao acaso** (MCAR), **por motivo observável** (MAR) ou **por motivo ligado ao próprio valor** (MNAR; o viés é inevitável). Em séries temporais, importa se a série é um **fluxo suave** (câmbio: interpolar faz sentido) ou um **degrau** (meta da Selic: repetir o último valor é o certo):

```
rng_l = np.random.default_rng(3)
verdadeira = sel_l.loc["2025-01-01":"2025-12-31"]
com_buracos = verdadeira.copy()
com_buracos[rng_l.random(len(com_buracos)) < 0.2] = np.nan
for nome_m, serie_m in {"ffill": com_buracos.ffill(), "interpolate":
com_buracos.interpolate()}.items():
    print(f"{nome_m:<12} erro absoluto médio por dia = {(serie_m -
verdadeira).abs().mean():.4f}")
print(f"Buracos: {com_buracos.isna().sum()} de {len(com_buracos)} dias")

base_cow = pd.DataFrame({"pais": ["Brasil", "Chile", "Peru"], "pib": [2.2, 3.1, 2.5]})
with pd.option_context("mode.copy_on_write", True):
    base2 = base_cow.copy()
    base2.loc[base2["pib"] > 2.3, "pib"] = 99                          # atribuição em UMA
chamada, com .loc
print("Atribuição segura com .loc:", base2["pib"].tolist())
```

```
ffill        erro absoluto médio por dia = 0.0000
interpolate  erro absoluto médio por dia = 0.0041
Buracos: 64 de 365 dias
Atribuição segura com .loc: [2.2, 99.0, 99.0]
```

Parte III — Visualização de Dados

Capítulo 13 — matplotlib

13.1 Filosofia

matplotlib tem duas APIs: - **pyplot** (`plt.plot()`) — rápida, para exploração - **OOP** (`fig, ax = plt.subplots()`) — explícita, para gráficos finais

Sempre prefira a OOP.

13.2 Gráfico de linha

```
import matplotlib.pyplot as plt
from bcb import sgs

ipca = sgs.get({"IPCA": 433}, start="2020-01-01")

fig, ax = plt.subplots(figsize=(12, 5))
ax.plot(ipca.index, ipca["IPCA"], color="crimson", linewidth=1.5)
ax.set_title("IPCA - Variação Mensal (%)", fontsize=14)
ax.set_ylabel("%")
ax.grid(True, alpha=0.3)
fig.tight_layout()
```

13.3 Parâmetros do plot

```
ax.plot(x, y,
    color="crimson",        # nome, hex (#FF5733), RGB ((1, 0, 0))
    linewidth=2,            # espessura
    linestyle="-",          # "-" sólida, "--" tracejada, ":" pontilhada,
                            # "-." traço-ponto
    marker="o",             # "o" círculo, "s" quadrado, "^" triângulo
    markersize=6,
    alpha=0.8,              # transparência
    label="IPCA"            # legenda
)
```

13.4 Personalização completa

```
fig, ax = plt.subplots(figsize=(12, 5.5))

ax.plot(ipca.index, ipca["IPCA"], color="crimson", linewidth=1.2, label="IPCA")
ax.axhline(0, color="gray", linewidth=0.5)
ax.axhline(ipca.mean().iloc[0], color="blue", linestyle="--", alpha=0.5, label="Média")

ax.set_title("Brasil: IPCA Mensal (2020-2026)", fontsize=14, fontweight="bold")
ax.set_ylabel("Variação mensal (%)", fontsize=12)
ax.legend(frameon=True, fancybox=True, shadow=True, fontsize=10)

ax.grid(True, alpha=0.3, linestyle=":")
ax.set_axisbelow(True)

ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)

fig.tight_layout()
fig.savefig("ipca_final.png", dpi=300, bbox_inches="tight")
```

13.5 Dois eixos Y

```
from bcb import sgs

ipca = sgs.get({"IPCA": 433}, start="2020-01-01")
selic = sgs.get({"SELIC": 11}, start="2020-01-01")

fig, ax1 = plt.subplots(figsize=(12, 5))

ax1.plot(ipca.index, ipca["IPCA"], color="crimson", linewidth=1.5, label="IPCA")
ax1.set_ylabel("IPCA (%)", color="crimson", fontsize=12)
ax1.tick_params(axis="y", labelcolor="crimson")

ax2 = ax1.twinx()
ax2.plot(selic.index, selic["SELIC"], color="navy", linewidth=1.5, label="SELIC")
ax2.set_ylabel("SELIC (%)", color="navy", fontsize=12)
ax2.tick_params(axis="y", labelcolor="navy")

ax1.set_title("IPCA e SELIC (2020-2026)", fontsize=14)
ax1.grid(True, alpha=0.3)
fig.tight_layout()
```

13.6 Subplots

```
fig, axes = plt.subplots(2, 2, figsize=(14, 8))
axes = axes.flatten()

# Gráfico 1
axes[0].plot(ipca.index, ipca["IPCA"], color="crimson")
axes[0].set_title("IPCA")

# Gráfico 2
axes[1].plot(selic.index, selic["SELIC"], color="navy")
axes[1].set_title("SELIC")

# Gráfico 3
axes[2].hist(ipca["IPCA"], bins=30, color="steelblue", edgecolor="black")
axes[2].set_title("Distribuição IPCA")

# Gráfico 4
axes[3].boxplot(ipca["IPCA"])
axes[3].set_title("Boxplot IPCA")

for ax in axes:
    ax.grid(True, alpha=0.3)

fig.tight_layout()
```

13.7 Fill between

```
fig, ax = plt.subplots(figsize=(12, 5))
ax.plot(ipca.index, ipca["IPCA"], color="crimson", linewidth=1.5)
ax.fill_between(ipca.index, ipca["IPCA"], 0,
                where=(ipca["IPCA"] > 0).values,
                color="green", alpha=0.1, label="Inflação positiva")
ax.fill_between(ipca.index, ipca["IPCA"], 0,
                where=(ipca["IPCA"] < 0).values,
                color="red", alpha=0.1, label="Deflação")
ax.legend()
ax.grid(True, alpha=0.3)
```

Capítulo 14 — seaborn

14.1 Por que seaborn

Gráficos estatísticos mais bonitos com menos código.

```
import seaborn as sns

sns.set_theme(style="whitegrid")
```

14.2 Regplot — scatter + regressão

```
sns.regplot(data=df, x="educacao", y="pib_per_capita",
            scatter_kws={"alpha": 0.6}, line_kws={"color": "red"})
```

14.3 Pairplot — correlações de uma vez

```
sns.pairplot(df[["pib", "inflacao", "desemprego", "populacao"]])
```

14.4 Boxplot comparativo

```
sns.boxplot(data=df, x="regiao", y="pib")
```

Capítulo 15 — Gráficos para Economia

15.1 Publication-ready

```
import matplotlib.pyplot as plt
import matplotlib.ticker as ticker
from bcb import sgs
import seaborn as sns

sns.set_theme(style="whitegrid")

ipca = sgs.get({"IPCA": 433}, start="2000-01-01")
selic = sgs.get({"SELIC": 11}, start="2000-01-01")

fig, ax = plt.subplots(figsize=(12, 5.5))

ax.plot(ipca.index, ipca["IPCA"], color="#CC3311",
        linewidth=1.2, label="IPCA", alpha=0.85)
ax.plot(selic.index, selic["SELIC"], color="#0077BB",
        linewidth=1.2, label="SELIC", alpha=0.85)

ax.fill_between(ipca.index, ipca["IPCA"], 0,
                alpha=0.05, color="#CC3311")

ax.axhline(0, color="gray", linewidth=0.5)
ax.set_title("Brasil: Taxa SELIC e Inflação (IPCA) — 2000 a 2026",
             fontsize=14, fontweight="bold", pad=12)
ax.set_ylabel("Percentual (%)", fontsize=12)
ax.legend(fontsize=11, frameon=True, fancybox=True)

ax.xaxis.set_major_locator(ticker.MultipleLocator(2))
ax.xaxis.set_major_formatter(ticker.DateFormatter("%Y"))
```

```
ax.grid(True, alpha=0.25, linestyle=":")
ax.set_axisbelow(True)

ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)

fig.tight_layout()
fig.savefig("publication_ready.png", dpi=300, bbox_inches="tight")
plt.close()
```

15.2 Cores seguras para daltônicos

```
cores = ["#0077BB", "#33BBEE", "#EE7733",
         "#CC3311", "#009988", "#BBBBBB"]
```

Aprofundando a teoria — tinta de dados, hierarquia visual e a ciência da percepção
Edward Tufte propôs a **razão tinta-dados**: o gráfico ideal tem o máximo de dados e o mínimo de enfeite. Estudos de Cleveland e McGill ordenaram as tarefas visuais por precisão (posição em escala comum > comprimento > ângulo > área > cor), o que explica por que barras superam pizzas. Duas regras de ouro: **barras sempre a partir de zero** (o comprimento codifica o valor) e **nunca dependa só da cor** (acrescente traço, marcador e rótulo direto). A figura abaixo reúne os elementos de uma boa figura e o contraste entre eixo cortado e eixo honesto:

```
import matplotlib.dates as mdates
fig, (ax1, ax2, ax3) = plt.subplots(1, 3, figsize=(11, 3.3))
ax1.axhspan(1.5, 4.5, color="#cfe3f5", alpha=0.6, label="Tolerância (1,5% a 4,5%)")
ax1.axhline(3.0, color="#1d4f91", linestyle="--", linewidth=1, label="Meta (3%)")
ax1.plot(ip_l.index, ip_l.values, color="#a33a1c", linewidth=2, label="IPCA 12m")
ax1.set_title("Título que informa", loc="left", fontsize=10, fontweight="bold");
ax1.legend(fontsize=7, frameon=False); ax1.set_ylim(0, 7)
anual = sel_l.groupby(sel_l.index.year).mean().loc[2025:2026]
for ax, lim, tit in ((ax2, (14.3, 14.6), "Eixo cortado"), (ax3, (0, 16), "Eixo a partir de
zero")):
    ax.bar(anual.index.astype(str), anual.values, color="#56B4E9", width=0.5);
ax.set_ylim(*lim)
    ax.set_title(tit, loc="left", fontsize=10, fontweight="bold")
for ax in (ax1, ax2, ax3):
    ax.spines[["top", "right"]].set_visible(False)
ax1.xaxis.set_major_locator(mdates.YearLocator());
ax1.xaxis.set_major_formatter(mdates.DateFormatter("%Y"))
fig.text(0.01, 0.01, "Fonte: Banco Central do Brasil (SGS); elaboração própria.",
fontsize=7, color="gray")
fig.tight_layout(rect=(0, 0.04, 1, 1)); fig.savefig("figurasli/boas_figuras.png", dpi=160);
plt.close(fig)
efeito = (anual.iloc[-1] / anual.iloc[0] - 1) * 100
cortado = ((anual.iloc[-1] - 14.3) / (anual.iloc[0] - 14.3) - 1) * 100
print(f"Selic média {anual.round(2).to_dict()} | diferença real {efeito:+.1f}% | no eixo
cortado parece {cortado:+.0f}%")
```

```
Selic média {2025: 14.42, 2026: 14.56} | diferença real +1.0% | no eixo cortado parece
+123%
```

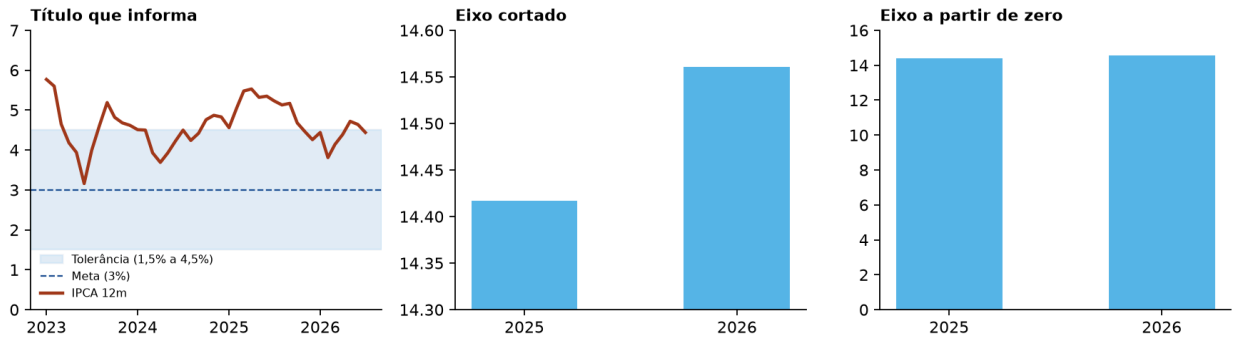


Figura 1: Elementos de uma boa figura (esquerda) e a mesma diferença de 1% com eixo cortado (centro) e honesto (direita).

Exemplo resolvido — heatmap: escala divergente para correlações
Para uma matriz de correlação, a escala de cor deve ser **divergente**, **simétrica** (-1 a $+1$) e centrada em zero; o arco-íris (`jet`) cria faixas falsas e é ilegível para daltônicos.

```
corr = gr_l.corr()
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(9.5, 3.8))
for ax, cmap, tit in ((ax1, "jet", "Errado: jet"), (ax2, "RdBu_r", "Certo: divergente,
centrada em zero")):
    im = ax.imshow(corr, cmap=cmap, vmin=-1, vmax=1)
    ax.set_title(tit, loc="left", fontsize=10, fontweight="bold")
    ax.set_xticks(range(len(corr))); ax.set_xticklabels(corr.columns, rotation=40,
ha="right", fontsize=7)
    ax.set_yticks(range(len(corr))); ax.set_yticklabels(corr.columns, fontsize=7)
    fig.colorbar(im, ax=ax, fraction=0.046)
fig.tight_layout(); fig.savefig("figurasli/heatmap.png", dpi=160); plt.close(fig)
pares = corr.where(np.triu(np.ones(corr.shape), 1).astype(bool)).stack().sort_values()
print(f"Maior correlação entre grupos do IPCA: {pares.index[-1]} = {pares.iloc[-1]:.2f} |
menor: {pares.index[0]} = {pares.iloc[0]:.2f}")
```

```
Maior correlação entre grupos do IPCA: ('Livres', 'Comercializaveis') = 0.92 | menor:
('Administrados', 'Livres') = -0.83
```

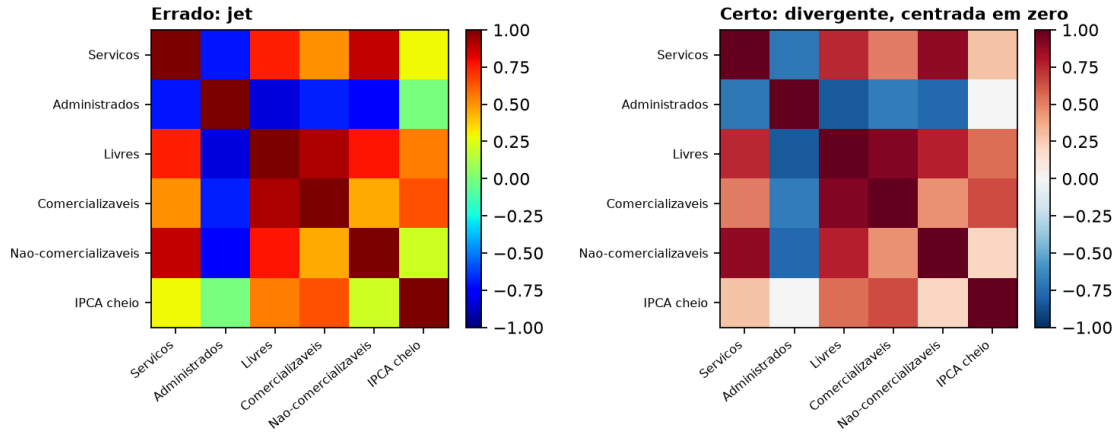


Figura 2: A mesma matriz de correlação (grupos do IPCA): arco-íris à esquerda, escala divergente à direita.

Parte IV — Econometria

Capítulo 16 — Regressão Linear

16.1 Modelo OLS com statsmodels

```
import statsmodels.api as sm
import numpy as np
import pandas as pd

# Dados: Consumo Keynesiano
np.random.seed(42)
renda = np.arange(1000, 11000, 1000)
consumo = 200 + 0.8 * renda + np.random.normal(0, 80, len(renda))

df = pd.DataFrame({"renda": renda, "consumo": consumo})

# Estimar: consumo = β₀ + β₁ * renda
X = sm.add_constant(df["renda"])
y = df["consumo"]

modelo = sm.OLS(y, X).fit()
print(modelo.summary())
```

16.2 Extraindo resultados

```
modelo.params             # [const, renda]
modelo.pvalues            # p-valores
modelo.rsquared           # R²
modelo.rsquared_adj       # R² ajustado
modelo.resid              # resíduos
modelo.fittedvalues       # valores preditos
modelo.conf_int()         # intervalo de confiança (95%)
modelo.bse                # erro padrão
modelo.aic                # AIC
modelo.bic                # BIC
modelo.nobs               # número de observações
modelo.df_resid           # graus de liberdade
modelo.fvalue             # estatística F
modelo.f_pvalue           # p-valor do F
```

16.3 Regressão múltipla

```
# Consumo ~ renda + riqueza + juros
df["riqueza"] = renda * 3 + np.random.normal(0, 500, len(renda))
df["juros"] = 5 + np.random.normal(0, 1, len(renda))

X = sm.add_constant(df[["renda", "riqueza", "juros"]])
y = df["consumo"]

modelo = sm.OLS(y, X).fit()
print(modelo.summary())
```

16.4 Regressão com fórmula

```
import statsmodels.formula.api as smf

modelo = smf.ols("consumo ~ renda + riqueza + juros", data=df).fit()
```

16.5 Curva de Phillips com dados reais

```
from bcb import sgs

# IPCA e Desemprego
ipca = sgs.get({"IPCA": 433}, start="2012-01-01")
desemprego = sgs.get({"DESEMPREGO": 24369}, start="2012-01-01")

df = pd.merge(ipca, desemprego, on="Date").dropna()
df.columns = ["ipca", "desemprego"]

# Com lag da inflação
df["ipca_lag1"] = df["ipca"].shift(1)
df = df.dropna()

modelo = smf.ols("ipca ~ desemprego + ipca_lag1", data=df).fit()
print(modelo.summary())
print(f"\nCoef. desemprego: {modelo.params['desemprego']:.4f}")
print(f"P-valor: {modelo.pvalues['desemprego']:.4f}")
```

Capítulo 17 — Diagnóstico e Validação

17.1 Normalidade dos resíduos

```
from scipy import stats

residuos = modelo.resid

# Teste Jarque-Bera
jb = stats.jarque_bera(residuos)
print(f"Jarque-Bera: {jb[0]:.4f} (p-valor: {jb[1]:.4f})")
# H0: resíduos normais. Se p > 0.05, não rejeita normalidade

# Teste de Shapiro-Wilk (mais poderoso)
sw = stats.shapiro(residuos)
print(f"Shapiro-Wilk: {sw[0]:.4f} (p-valor: {sw[1]:.4f})")

# Q-Q plot
fig, ax = plt.subplots(figsize=(6, 6))
stats.probplot(residuos, dist="norm", plot=ax)
```

17.2 Heterocedasticidade

```
from statsmodels.stats.diagnostic import het_breuschpagan

bp = het_breuschpagan(residuos, modelo.model.exog)
nomes = ["LM Stat", "LM p-valor", "F Stat", "F p-valor"]
print(dict(zip(nomes, [round(v, 4) for v in bp])))

# H0: homocedasticidade. Se p < 0.05, há heterocedasticidade
```

17.3 Autocorrelação

```
from statsmodels.stats.stattools import durbin_watson

dw = durbin_watson(residuos)
print(f"Durbin-Watson: {dw:.4f}")
# 2 = sem autocorrelação
```

```
# < 1.5 = autocorrelação positiva
# > 2.5 = autocorrelação negativa

# Ljung-Box (teste para autocorrelação em lags específicos)
from statsmodels.stats.diagnostic import acorr_ljungbox
lb = acorr_ljungbox(residuos, lags=[6, 12, 24])
print(lb)
```

17.4 Multicolinearidade (VIF)

```
from statsmodels.stats.outliers_influence import variance_inflation_factor

X = df[["desemprego", "ipca_lag1"]]
X = sm.add_constant(X)

vif = pd.DataFrame()
vif["variavel"] = X.columns
vif["VIF"] = [variance_inflation_factor(X.values, i) for i in range(X.shape[1])]
print(vif)
# VIF > 10 indica multicolinearidade severa
```

17.5 Erros padrão robustos

```
# HC0 = White, HC1 = ajustado por graus de liberdade
modelo_robusto = modelo.get_robustcov_results(cov_type="HC1")
print(modelo_robusto.summary())
```

Capítulo 18 — Séries Temporais

18.1 Estacionariedade (ADF)

```
from statsmodels.tsa.stattools import adfuller

result = adfuller(serie.dropna())
print(f"ADF: {result[0]:.4f}")
print(f"p-valor: {result[1]:.4f}")
print(f"Valores críticos: {result[4]}")

if result[1] < 0.05:
    print("Série estacionária")
else:
    print("Série não estacionária — precisa diferenciar")
```

18.2 ACF e PACF

```
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf

fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(12, 8))
plot_acf(serie, lags=24, ax=ax1)
plot_pacf(serie, lags=24, ax=ax2)
```

18.3 ARIMA

```
from statsmodels.tsa.arima.model import ARIMA

modelo = ARIMA(serie, order=(p, d, q))
```

```
resultado = modelo.fit()
print(resultado.summary())

# Previsão
forecast = resultado.forecast(steps=12)
conf_int = resultado.get_forecast(12).conf_int()
```

18.4 SARIMA (sazonal)

```
from statsmodels.tsa.statespace.sarimax import SARIMAX

# SARIMA(p,d,q)(P,D,Q,s) — s=12 para mensal
modelo = SARIMAX(serie,
                 order=(1, 1, 1),
                 seasonal_order=(1, 1, 1, 12),
                 enforce_stationarity=False,
                 enforce_invertibility=False)
resultado = modelo.fit(disp=False)
```

18.5 Seleção automática de ordem

```
import itertools

p = range(0, 4)
d = range(0, 2)
q = range(0, 4)

best_aic = float("inf")
best_order = None

for pd_ in p:
    for d_ in d:
        for q_ in q:
            try:
                modelo = ARIMA(serie, order=(pd_, d_, q_))
                resultado = modelo.fit()
                if resultado.aic < best_aic:
                    best_aic = resultado.aic
                    best_order = (pd_, d_, q_)
            except:
                continue

print(f"Melhor ARIMA{best_order}, AIC={best_aic:.2f}")
```

18.6 Cointegração

```
from statsmodels.tsa.stattools import coint

# Teste Engle-Granger
score, pvalue, _ = coint(y1, y2)
print(f"p-valor cointegração: {pvalue:.4f}")

if pvalue < 0.05:
    print("Séries são cointegradas (relação de longo prazo)")
```

18.7 Modelo VAR

```
from statsmodels.tsa.api import VAR

modelo = VAR(df)
```

```python
resultado = modelo.fit(maxlags=6, ic="aic")
print(f"Lags ótimos: {resultado.k_ar}")

# Função impulso-resposta
irf = resultado.irf(12)
irf.plot()

# Previsão
forecast = resultado.forecast(df.values[-resultado.k_ar:], steps=6)
```

18.8 Causalidade de Granger

```python
from statsmodels.tsa.stattools import grangercausalitytests

gc = grangercausalitytests(df[["y", "x"]], maxlag=6, verbose=False)
for lag in range(1, 7):
    p = gc[lag][0]["ssr_ftest"][1]
    print(f"Lag {lag}: p={p:.4f} {'Causa' if p < 0.05 else 'Não causa'}")
```

Capítulo 19 — Dados de Painel

19.1 Estrutura de painel

Dados que acompanham as mesmas unidades (países, estados) ao longo do tempo.

19.2 Pooled OLS

```python
# Ignora estrutura de painel
X = sm.add_constant(df[["x1", "x2"]])
modelo = sm.OLS(y, X).fit()
```

19.3 Efeitos Fixos

```python
from linearmodels.panel import PanelOLS

df = df.set_index(["pais", "ano"])

modelo = PanelOLS.from_formula(
    "y ~ x1 + x2 + EntityEffects",
    data=df
).fit()
```

19.4 Efeitos Aleatórios

```python
from linearmodels.panel import RandomEffects

modelo = RandomEffects.from_formula("y ~ x1 + x2", data=df).fit()
```

19.5 Teste de Hausman

```python
from linearmodels.panel import compare

fe = PanelOLS.from_formula("y ~ x1 + EntityEffects", data=df).fit()
re = RandomEffects.from_formula("y ~ x1", data=df).fit()

print(compare({"FE": fe, "RE": re}))
```

```
# Se p < 0.05: usar efeitos fixos
# Se p > 0.05: usar efeitos aleatórios
```

Aprofundando a teoria — o que significa "não viesado" e o que acontece quando as premissas falham
O estimador OLS $\hat{\beta} = (X^T X)^{-1} X^T y$ é uma variável aleatória: outra amostra, outro valor. "Não viesado" significa que, na média sobre infinitas amostras, $E\left[\hat{\beta}\right] = \beta$. Simulamos para *ver*: (1) o OLS acerta em média e o erro-padrão reproduz a dispersão real; (2) omitir uma variável correlacionada com o regressor gera viés $\beta_2 \cdot \frac{\mathrm{Cov}(x,z)}{\mathrm{Var}(x)}$; (3) regredir passeios aleatórios independentes produz "significância" espúria; (4) com heterocedasticidade, o erro-padrão clássico perde cobertura.

```
import statsmodels.api as sm
rng_o = np.random.default_rng(1)
b_hat = []
for _ in range(2000):
    x = rng_o.uniform(2, 20, 60); y = 1.5 + 0.8 * x + rng_o.normal(0, 1.2, 60)
    b_hat.append(np.polyfit(x, y, 1)[0])
print(f"(1) Verdadeiro 0.8 | média de 2000 estimativas: {np.mean(b_hat):.4f} | desvio
padrão real: {np.std(b_hat, ddof=1):.4f}")

rng_v = np.random.default_rng(4)
z = rng_v.normal(size=5000); x = 0.6 * z + rng_v.normal(scale=0.8, size=5000); y = 1 + 2 *
x + 3 * z + rng_v.normal(size=5000)
curta = sm.OLS(y, sm.add_constant(x)).fit()
delta = np.cov(x, z)[0, 1] / np.var(x, ddof=1)
print(f"(2) Modelo curto: b1 = {curta.params[1]:.3f} (verdadeiro 2) | viés previsto = {3 *
delta:.3f} | observado = {curta.params[1] - 2:.3f}")

rng_r = np.random.default_rng(50)
sig = 0
for _ in range(1000):
    a_ = np.cumsum(rng_r.normal(size=200)); b_ = np.cumsum(rng_r.normal(size=200))
    sig += sm.OLS(a_, sm.add_constant(b_)).fit().pvalues[1] < 0.05
print(f"(3) Passeios aleatórios independentes 'significativos' a 5%: {sig / 1000:.0%}
(esperado 5%)")

rng_h = np.random.default_rng(12)
cob_c = cob_r = 0
for _ in range(2000):
    x = rng_h.uniform(1, 10, 100); y = 1 + x + rng_h.normal(0, 0.5 * x)
    aj = sm.OLS(y, sm.add_constant(x)).fit()
    cob_c += aj.conf_int()[1, 0] <= 1 <= aj.conf_int()[1, 1]
    rb = aj.get_robustcov_results(cov_type="HC3")
    cob_r += rb.conf_int()[1, 0] <= 1 <= rb.conf_int()[1, 1]
print(f"(4) Cobertura do IC de 95%: clássico = {cob_c / 2000:.1%} | robusto HC3 = {cob_r /
2000:.1%}")
```

```
(1) Verdadeiro 0.8 | média de 2000 estimativas: 0.7992 | desvio padrão real: 0.0305
(2) Modelo curto: b1 = 3.757 (verdadeiro 2) | viés previsto = 1.774 | observado = 1.757
(3) Passeios aleatórios independentes 'significativos' a 5%: 85% (esperado 5%)
(4) Cobertura do IC de 95%: clássico = 93.9% | robusto HC3 = 95.7%
```

Parte V — Projetos

Capítulo 20 — Projeto: Análise Macroeconômica do Brasil

Script completo que baixa dados do BCB, processa e gera relatório.

```python
from bcb import sgs
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.ticker as ticker

codigos = {
    "IPCA": 433,
    "SELIC": 11,
    "DESEMPREGO": 24369,
    "CAMBIO": 1,
    "PIB": 4380
}

dados = sgs.get(codigos, start="2010-01-01")

# Processar
dados["IPCA_ACUM"] = dados["IPCA"].rolling(12).sum()
dados["JURO_REAL"] = dados["SELIC"] - dados["IPCA_ACUM"]
dados["CAMBIO_VAR"] = dados["CAMBIO"].pct_change() * 100

# Relatório
print("=" * 60)
print("RELATÓRIO MACROECONÔMICO - BRASIL")
print("=" * 60)
ultimos = dados.tail(1)
print(f"Data: {ultimos.index[0].date()}")
print(f"IPCA mensal: {ultimos['IPCA'].values[0]:.2f}%")
print(f"IPCA acum. 12m: {ultimos['IPCA_ACUM'].values[0]:.2f}%")
print(f"SELIC: {ultimos['SELIC'].values[0]:.2f}%")
print(f"Juro real: {ultimos['JURO_REAL'].values[0]:.2f}%")
print(f"Desemprego: {ultimos['DESEMPREGO'].values[0]:.2f}%")
print(f"Câmbio: {ultimos['CAMBIO'].values[0]:.2f}")

# Gráficos
fig, axes = plt.subplots(3, 2, figsize=(14, 10))
series = [
    ("IPCA (mensal)", "crimson"),
    ("SELIC", "navy"),
    ("IPCA Acum. 12m", "darkred"),
    ("Desemprego", "darkgreen"),
    ("Câmbio (USD/BRL)", "purple"),
    ("Juro Real", "orange")
]
for ax, (titulo, cor) in zip(axes.flatten(), series):
    ax.plot(dados.index, dados[titulo], color=cor, linewidth=1.2)
    ax.set_title(titulo, fontsize=12)
    ax.grid(True, alpha=0.3)
fig.tight_layout()
fig.savefig("dashboard.png", dpi=200)
```

Capítulo 21 — Projeto: Previsão de Inflação com SARIMA

```python
from bcb import sgs
import matplotlib.pyplot as plt
from statsmodels.tsa.statespace.sarimax import SARIMAX

ipca = sgs.get({"IPCA": 433}, start="2005-01-01")
serie = ipca["IPCA"]

train = serie[:-12]
test = serie[-12:]

modelo = SARIMAX(train,
                 order=(1, 1, 1),
                 seasonal_order=(1, 1, 1, 12),
                 enforce_stationarity=False,
                 enforce_invertibility=False)
resultado = modelo.fit(disp=False)

forecast = resultado.forecast(steps=12)

from sklearn.metrics import mean_absolute_error
mae = mean_absolute_error(test, forecast)
print(f"MAE: {mae:.2f} pp")

fig, ax = plt.subplots(figsize=(12, 5))
ax.plot(train.index[-60:], train[-60:], label="Histórico")
ax.plot(test.index, test, label="Real", color="green")
ax.plot(forecast.index, forecast, label="Previsão", color="red", linestyle="--")
ax.fill_between(forecast.index,
                resultado.get_forecast(12).conf_int().iloc[:, 0],
                resultado.get_forecast(12).conf_int().iloc[:, 1],
                alpha=0.2, color="red")
ax.legend()
ax.grid(True, alpha=0.3)
fig.tight_layout()
fig.savefig("previsao.png", dpi=150)
```

Capítulo 22 — Projeto Final: Estrutura de TCC

```
meu_tcc/
├── dados/              # Dados brutos (imutáveis)
├── scripts/            # Código numerado
│   ├── 01_baixar.py
│   ├── 02_limpeza.py
│   ├── 03_descritiva.py
│   ├── 04_modelos.py
│   └── 05_graficos.py
├── output/             # Resultados
│   ├── tabelas/
│   └── graficos/
├── environment.yml     # Ambiente replicável
└── README.md
```

Exemplo resolvido — laboratório de projeto: decompondo a Selic implícita da regra de Taylor com dados reais
A regra do repositório é $i^* = r^* + \pi + 0{,}5\ (\pi - \pi^*)$: juro neutro mais compensação da inflação mais reação ao desvio da meta. O **princípio de Taylor** exige coeficiente total sobre a inflação maior que 1 (aqui, 1,5), para que o juro real suba com a inflação.

```
ult = tay_l.iloc[-1]
print(f"{tay_l.index[-1]:%m/%Y}: Selic implícita {ult['taylor_selic']:.2f}% = neutro
{ult['r_star']:.2f} + inflação {ult['ipca12m']:.2f} + reação {0.5 * (ult['ipca12m'] -
ult['meta']):.2f}")
print(f"Selic efetiva {ult['selic']:.2f}% | gap {ult['gap_pp']:+.2f} p.p. | gap médio da
série: {tay_l['gap_pp'].mean():+.2f} p.p.")
import sys, platform
print(f"Ambiente: Python {sys.version.split()[0]} | pandas {pd.__version__} | numpy
{np.__version__} | statsmodels {sm.__version__}")
```

```
07/2026: Selic implícita 10.16% = neutro 5.00 + inflação 4.44 + reação 0.72
Selic efetiva 14.25% | gap +4.09 p.p. | gap médio da série: +3.04 p.p.
Ambiente: Python 3.12.10 | pandas 2.2.3 | numpy 2.5.4 | statsmodels 0.15.0
```

Atenção — erro comum
A regra de Taylor "simples" é uma **referência de calibragem**, não um veredito: o Copom também olha o hiato do produto, as expectativas e o cenário externo, e a taxa neutra r^* é inobservável. Um gap positivo persistente indica política mais apertada que a regra sugere, mas não prova erro.

Apêndices

Apêndice A — Guia rápido de consulta

Operação	Código
Abrir CSV	`pd.read_csv("arquivo.csv")`
Filtrar	`df[df["col"] > 0]`
Agrupar	`df.groupby("x")["y"].mean()`
Juntar	`pd.merge(a, b, on="x")`
Lag	`df["x"].shift(1)`
Média móvel	`df["x"].rolling(3).mean()`
Regressão	`sm.OLS(y, X).fit()`
ARIMA	`ARIMA(y, order=(p,d,q)).fit()`
ADF	`adfuller(serie)`
Gráfico	`ax.plot(x, y)`

Apêndice B — Códigos BCB SGS mais usados

Série	Código
IPCA (mensal)	433
IPCA (acum. 12m)	13522

Série	Código
SELIC (meta)	11
SELIC (efetiva)	1178
Câmbio (USD)	1
Desemprego	24369
PIB mensal	4380
Produção industrial	21855
Dívida líquida (%PIB)	4538
Resultado primário	4501

Apêndice C — Soluções dos Exercícios

Capítulo 2

1. `5000 * (1 + 0.015) ** 24`
2. `1000 * 5.45`
3. False, True, True
4. `meta_descumprida = inflacao > 3.5`

Capítulo 3

1.

```
pib = [2.2, -1.8, 3.1, 2.8, -0.5]
positivos = [x for x in pib if x > 0]
print(len(positivos), sum(positivos) / len(positivos))
```

2.

```
paises = {"Brasil": {"inflacao": 4.5}, "Argentina": {"inflacao": 98.0}}
print(abs(paises["Brasil"]["inflacao"] - paises["Argentina"]["inflacao"]))
```

3.

```
ipca = [0.5, 0.7, 0.88, 0.67, 0.58, 0.16]
print(ipca[:3], ipca[-3:], ipca[::-1])
```

4. EUR é a maior. 3 moedas abaixo de R$ 1. `moedas["JPY"] = 0.036`

Capítulo 4

1. Loop com if/elif/else.

2.

```
saldo = 5000
for mes in range(1, 37):
    saldo *= 1.008
    if mes % 6 == 0:
        print(f"Mês {mes}: R$ {saldo:.2f}")
```

3.

```
saldo = 1000
meses = 0
```

```python
while saldo < 2000:
    saldo *= 1.012
    meses += 1
print(meses)
```

4.

```python
for v in [0.5, 0.7, 0.88, 0.67, 0.58, 0.16]:
    if v > 1:
        print(v)
        break
```

Capítulo 5

1.

```python
def inflacao_acumulada(taxas):
    acum = 1
    for t in taxas:
        acum *= (1 + t/100)
    return (acum - 1) * 100
```

2.

```python
def curva_laffer(taxa, base=1000):
    return taxa * base * (1 - taxa)
# Ótimo: taxa = 0.5
```

3.

```python
def classificar(pib, inf):
    if pib > 2 and inf < 5: return "Bom"
    elif pib < 0 and inf > 8: return "Crítico"
    else: return "Atenção"
```

4.

```python
sorted(paises, key=lambda x: x[1], reverse=True)
```

Fim do livro. Lembre-se: teoria sem prática é inútil. Abra o terminal e execute cada exemplo.

Apêndice D — Erros Comuns e Teste Rápido

#	Erro	Como evitar
1	Comparar floats com `==`	`math.isclose` ou tolerância
2	Achar que `b = a` copia uma lista	`.copy()` ou `copy.deepcopy`
3	Usar `and`/`or` em máscaras do pandas	`&`, `|`, `~` com parênteses
4	Atribuição encadeada `df[m]["x"] = 0`	`df.loc[m, "x"] = 0`
5	Merge sem checar a chave	`is_unique`, `validate=`, contar linhas
6	Barras com eixo cortado; arco-íris (`jet`)	Zero na base; escala divergente centrada em zero
7	Interpretar correlação como causalidade	Mecanismo, confundidores, causalidade reversa

#	Erro	Como evitar
8	Regredir séries em nível sem testar raiz unitária	ADF; diferenciar ou testar cointegração
9	Erro-padrão clássico "por padrão"	Robusto (HC3) ou HAC
10	Previsão sem intervalo nem avaliação fora da amostra	Intervalo e treino/teste no tempo

Teste rápido (V ou F, com gabarito).

1. `0.1 + 0.2 == 0.3` é `True`. **Falso.** Use tolerância.
2. Uma cópia rasa de uma lista de listas protege as listas internas. **Falso.** Só `deepcopy` copia tudo.
3. `merge` com chave duplicada nas duas tabelas preserva o número de linhas. **Falso.** Multiplica.
4. `ffill` é adequado para a meta da Selic. **Verdadeiro.** É uma série em degraus.
5. A multicolinearidade viesa os coeficientes. **Falso.** Infla os erros-padrão.
6. Duas séries independentes com raiz unitária podem dar regressão "significativa". **Verdadeiro.** É a regressão espúria.
7. Em um gráfico de barras, o eixo Y pode começar em qualquer valor. **Falso.** Deve partir de zero.
8. Uma regra de Taylor com gap positivo prova que o Banco Central errou. **Falso.** É referência, com incerteza sobre r^*.